Practicum: Bepaling van de brandpuntsafstand van een onbekende lens

Leerdoelen

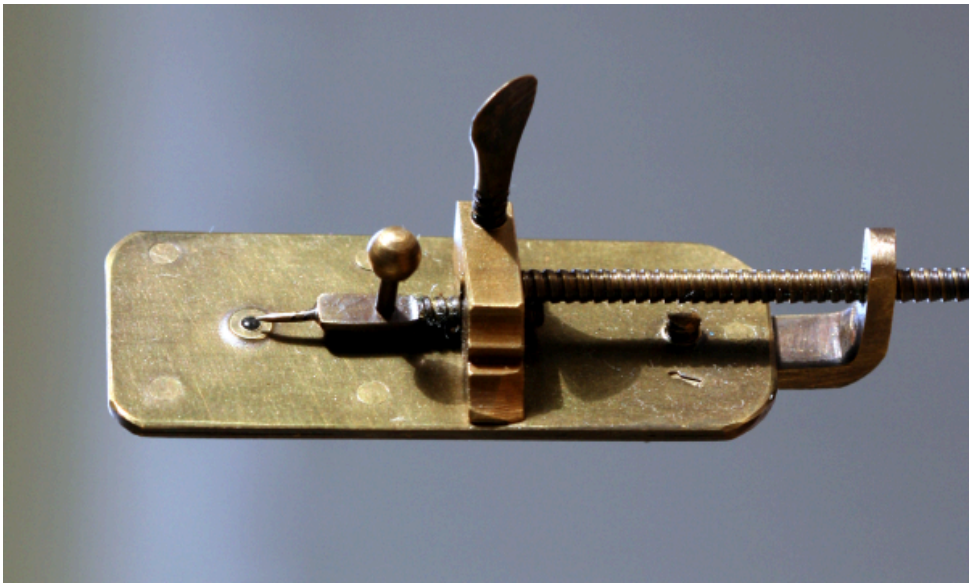
- ❒ Begrip van het belang van het gebruik van een (digitaal) labjournaal bij natuurkundig onderzoek.
- ❒ Zelfstandig kunnen bijhouden van een (digitaal) labjournaal in natuurkundepractica.
- ❒ Begrip van en toepassen van de lenzenformule.
- ❒ Basisvaardigheid in datamanipulatie en visualisatie in Python.

Veiligheid

1. We werken dit practicum met lenzen en LED lampen. Ga voorzichtig om met de lenzen en pas op voor de LED lampen, deze kunnen heet worden en mogen niet warm opgeborgen worden.
2. De LED lamp kan maximaal 12 volt aan!

Introductie

Al sinds de oudheid gebruiken mensen lenzen om beelden te vergroten of scherp te stellen. De oude Grieken en Romeinen kenden al glaskogeltjes die het licht bundelden, en in de middeleeuwen verschenen de eerste brillen. In de 17e eeuw werden brillen en lenzen steeds verfijnder, en in die tijd begon men microscopen en telescopen te ontwikkelen. Een van de bekendste Nederlanders, Antoni van Leeuwenhoek, bouwde met zelfgeslepen lenzen ontzettend sterke microscopen[1], zie Fig. 1.



Figuur 1: Een replica van één van de eerste microscopen van Antoni van Leeuwenhoek.

Met deze microscopen werden voor het eerst bacteriën, spiervezels en bloedcellen ontdekt. Zijn werk markeert het begin van de microscopie, en laat zien hoe een relatief eenvoudige lens tot grote wetenschappelijke ontdekkingen kan leiden.

De werking van een lens is in essentie eenvoudig: een lens buigt lichtstralen af doordat het licht bij de overgang van lucht naar glas (of andersom) van richting verandert. Hierdoor kunnen lichtstralen die uit één punt van een voorwerp komen, weer in één punt samenkomen: het beeldpunt. Hoe sterk een lens lichtstralen bundelt of spreidt, wordt bepaald door de *brandpuntsafstand* f.

De wiskundige relatie tussen voorwerpafstand v, beeldafstand b en brandpuntsafstand f staat bekend als de *lenzenformule*:

$$\frac{1}{f} = \frac{1}{v} + \frac{1}{b}.$$

In dit practicum ga je deze relatie zelf onderzoeken. Door voor verschillende voorwerpsafstanden v de bijbehorende beeldafstanden b te meten, kun je de brandpuntsafstand van een onbekende lens afleiden. Op deze manier volg je in zekere zin de voetsporen van Van Leeuwenhoek: met een lens de wereld op een nieuwe manier zichtbaar maken.

Voorbereidingsopdracht 1

1 Neem een laptop mee.

[1] Het vakmanschap van Antoni van Leeuwenhoek was zo uitzonderlijk, dat het tot voor kort een raadsel bleef hoe hij zulke krachtige microscopen kon bouwen. Pas enkele jaren geleden is dit mysterie deels ontrafeld in de nucleaire reactor in Delft (zie dit artikel).

2. Zorg voor een goede installatie van Python & Jupyter notebooks, zie practicumhandleiding week 1.

Opdracht 1(a)

Meet voor verschillende voorwerpsafstanden v de bijbehorende beeldafstanden b en vergrotingen.

1. Noteer bij elke meting ook de meetfout σ_b.
2. Leg uit hoe je de grootte van σ_b hebt bepaald. Hoe kun je deze nauwkeuriger bepalen?
3. Noteer deze gegevens in je Digitaal Labjournaal.

Tip! Gebruik de onderstaande code en plak deze in je labjournaal:

Data invoeren

```
import numpy as np

# meetwaarden
b = np.array([])
v = np.array([])
b_err = np.array([])
```

Opdracht 1(b)

Maak in Python een grafiek waarin je b uitzet tegen v. Voeg duidelijke labels en eenheden toe.
Tip! Gebruik de onderstaande code en plak deze in je labjournaal:

Grafiek maken

```
import matplotlib.pyplot as plt

# plotten
plt.errorbar(v, b, yerr=b_err, fmt='o')
plt.xlabel("v (m)")
plt.ylabel("b (m)")
plt.show()
```

Zoals je kunt zien in het figuur is er een duidelijk, maar soms ook verwarrend, verbanden tussen de beeldafstand en de voorwerpsafstand. De lenzenformule

$$\frac{1}{f} = \frac{1}{v} + \frac{1}{b}, \tag{1}$$

geeft echter niet direct het verband aan tussen b en v.

Opdracht 1(c)

1. Herschrijf de lensvergelijking zodat b wordt uitgedrukt als functie van v en f.
2. Gebruik onderstaande Python-code en vul deze correct aan.
3. Noteer je afleiding in je Digitaal Labjournaal.

Functie maken

```
def beeldafstand(v, f):
    return (...)
```

Opdracht 1(d)

1. Maak een array die loopt van 0 tot 1 meter in stapjes van 1 cm.
2. Teken de functie van de *beeldafstand* voor verschillende waarden van de brandpuntsafstand f. Welke waarde komt het meest overeen met de onbekende lens?

Lineaire span

```
v_fit = np.linspace(0, 1, 101) # in meters
# Gebruik voor elke gekozen f alleen v_fit > f (reeel beeld).
# Bij v = f is de beeldafstand niet eindig.
```

Het tekenen van verschillende waarden van *f* levert een eerste schatting op, maar houdt geen rekening met meetonzekerheden. Grotere afstanden betekenen niet automatisch nauwkeurigere metingen: schat de onzekerheid per meting. We gebruiken hier *lineariseren*: door zowel de beeldafstand als de voorwerpsafstand te inverteren, ontstaat een lineair verband tussen de nieuwe variabelen. Lineariseren is niet hetzelfde als inverteerbaarheid. Bij deze transformatie moeten ook de onzekerheden worden omgerekend.

Opdracht 1(e)

1. Maak een nieuwe grafiek waarbij je $1/b$ uitzet tegen $1/v$.
2. Schrijf de lenzenformule om in de vorm

$$y = ax + c, \qquad y = \frac{1}{b}, \quad x = \frac{1}{v}. \tag{2}$$

3. Maak een nieuwe functie genaamd *lineair*, met als invoer `inverse_v = 1/v` en parameter `f`. De uitvoer is `1/b`.
4. Wat valt je op aan de grafiek? Wat is de betekenis van het snijpunt met de y-as?
5. Voeg nu enkele lijnen toe met verschillende waarden van de brandpuntsafstand f.

Functie maken

```
def lineair(inverse_v, f):
    return (...)
```

Zoals je misschien hebt opgemerkt, ziet de grafiek er nu aanzienlijk beter uit. De vraag blijft echter wat het snijpunt met de y-as betekent en welke waarde hier precies bij hoort. Om dit op een natuurkundig correcte manier uit te werken, gebruiken natuurkundigen vaak modellen. In dit geval is dat natuurlijk de functie $lineair$.
Nu moeten we ons model nog toetsen (*fitten*!) aan de gemeten data. Dit is in principe een optimalisatieprobleem: we moeten de fitwaarde *f* zo kiezen dat de afwijkingen tussen model en meting zo klein

[2] Het maakt niet uit als deze stap nog onduidelijk is; het is een trucje dat tijdens wiskunde 2.1 verder wordt uitgelegd.

mogelijk zijn[2]. Hiervoor maken we gebruik van de rekenkracht van Python, met de wetenschappelijke module **scipy**.

Data fitten

```
from scipy.optimize import curve_fit

# Schatting van parameters
p0 = [1] # eerste schatting van f, 1 meter

# aanroepen fit
popt, pcov = curve_fit(lineair, 1/v, 1/b, sigma=(...), p0=p0,
                       absolute_sigma=True)

# resultaat printen
print("$f$ =", popt, "+-", np.sqrt(np.diag(pcov)))
```

Opdracht 1(f)

1. Bereken de fout op $\frac{1}{b}$, gebruik hiervoor de relatieve fout van b. Als dit niet lukt, sla deze stap dan over.
2. Bepaal f door middel van een fit, gebruik hiervoor de *scipy*-module. Mocht het bepalen van de fout in de vorige opdracht niet gelukt zijn, verwijder dan de argumenten `sigma=(...)` en `absolute_sigma=True`. De onzekerheid wordt dan geschat uit de residuen, onder de aanname van gelijke spreiding in `1/b`.
3. Wat is de gevonden waarde voor *f*, en welke onzekerheid hoort hierop?

Afronding

1. Zet de gevonden figuren in je digitaal labjournaal en maak een duidelijk onderschrift waarin je uitlegt wat de figuren laten zien.
2. Vul de Resultaat- en Discussie-sectie aan in je labjournaal. Hierin moet de uiteindelijke waarde van f (met onzekerheid) staan en een korte discussie of deze waarde realistisch en betrouwbaar is.

Appendix: data fitten met `curve_fit`

Met een fit bepaal je welke waarde van een modelparameter het best bij je meetpunten past. In dit practicum is die parameter de brandpuntsafstand f. Hieronder doorloop je de fit van begin tot eind, met dezelfde variabelen als in de opdrachten.

Werkwijze: voer de codeblokken op volgorde uit in één notebook. De voorbeeldmetingen dienen om de code te oefenen; vervang ze daarna door je eigen metingen en onzekerheden.

1. Zet je meetgegevens klaar

Gebruik voor v, b en `b_err` dezelfde lengte-eenheid: hier meter. Elke positie in de drie arrays hoort bij één meting. `b_err` bevat de positieve standaardonzekerheid van elke beeldafstand, dus geen percentages.

Meetgegevens — vervang deze door je eigen metingen

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

v = np.array([0.180, 0.220, 0.260, 0.300, 0.360, 0.440])
b = np.array([0.226, 0.182, 0.164, 0.149, 0.139, 0.129])
b_err = np.array([0.002, 0.002, 0.002, 0.002, 0.002, 0.002])

# Nieuwe variabelen voor de gelineariseerde grafiek
inverse_v = 1 / v
inverse_b = 1 / b
sigma_inverse_b = b_err / b**2
```

Bij de transformatie $y = \frac{1}{b}$ verandert ook de onzekerheid. Voor kleine relatieve onzekerheden geldt:

$$\sigma_y = \left|\frac{\mathrm{d}y}{\mathrm{d}b}\right|\sigma_b = \frac{\sigma_b}{b^2}.$$

De waarden van `inverse_v`, `inverse_b` en `sigma_inverse_b` hebben dus de eenheid m^{-1}. Gebruik bij de fit van $\frac{1}{b}$ de omgerekende onzekerheid `sigma_inverse_b`.

2. Vertaal de lenzenformule naar een model

Herschrijven van de lenzenformule geeft:

$$\frac{1}{b} = -\frac{1}{v} + \frac{1}{f}, \quad \text{dus} \quad y = -x + \frac{1}{f}.$$

De helling ligt in dit model vast op -1. Alleen f wordt door de fit bepaald. Het eerste argument van de Python-functie is de onafhankelijke variabele; daarna volgen de te bepalen parameters.

Het model — één onbekende parameter

```python
def lineair(inverse_v, f):
    return -inverse_v + 1 / f
```

3. Laat Python de beste parameter bepalen

Geef `curve_fit` het model, de x-waarden, de y-waarden en de onzekerheden in y. Met `p0` geef je een eerste schatting van de parameter. Gebruik bij je eigen metingen de schatting uit opdracht 1(d).

De fit uitvoeren

```
p0 = [0.10]  # startschatting van f in meter

popt, pcov = curve_fit(
    lineair,
    inverse_v,
    inverse_b,
    p0=p0,
    sigma=sigma_inverse_b,
    absolute_sigma=True,
)
```

- `lineair`: de functie die je fit. Geef de functienaam mee, zonder haakjes.
- `inverse_v` en `inverse_b`: de gemeten x- en y-waarden na de transformatie.
- `p0`: een lijst met één startwaarde per vrije parameter, in de volgorde van de functieargumenten.
- `sigma`: de standaardonzekerheden in de y-waarden. Punten met kleinere onzekerheid wegen zwaarder mee.
- `absolute_sigma=True`: gebruik de opgegeven standaardonzekerheden ook om de parameteronzekerheid te schatten.

Aanname: deze fit behandelt $\frac{1}{v}$ als exact. Is de onzekerheid in v niet verwaarloosbaar, dan is een methode nodig die ook x-onzekerheden meeneemt. Alleen `xerr` toevoegen aan een grafiek verandert de fit niet.

4. Lees de uitkomst en de onzekerheid uit

`curve_fit` geeft twee objecten terug. `popt` bevat de gevonden parameterwaarden. `pcov` is een matrix waarvan de diagonaal de geschatte varianties van die parameters bevat. De wortels daarvan geven de standaardonzekerheden.

Brandpuntsafstand met standaardonzekerheid

```
perr = np.sqrt(np.diag(pcov))
f_fit = popt[0]  # eerste en enige parameter
f_err = perr[0]

print(f"f = {f_fit:.4f} +/- {f_err:.4f} m")
```

Met de voorbeeldgegevens vind je ongeveer $f = (0.1001 \pm 0.0003)\,\mathrm{m}$. Het getal achter $\pm$ is een geschatte standaardonzekerheid, geen gegarandeerde grens. Rond in je verslag de onzekerheid af op één of twee significante cijfers en de waarde op dezelfde decimaal.

Geen meetonzekerheden beschikbaar? Zonder `sigma` en `absolute_sigma=True` wegen alle punten even zwaar. De parameteronzekerheid wordt dan uit de residuen geschat, onder de aanname van gelijke spreiding in $\frac{1}{b}$. Bij het inverteren is die aanname vaak niet passend; gebruik daarom waar mogelijk je gemeten onzekerheden.

5. Controleer de fit in een grafiek

Een parameterwaarde alleen vertelt nog niet of het model de data goed beschrijft. Teken daarom de meetpunten met foutbalken én de gevonden lijn. Bekijk ook de **residuen**: de gemeten y-waarde min de modelwaarde bij dezelfde x-waarde.

Meetpunten, fit en residuen

```
x_plot = np.linspace(inverse_v.min(), inverse_v.max(), 200)
residuen = inverse_b - lineair(inverse_v, f_fit)

fig, (ax, ax_res) = plt.subplots(
    2, 1, sharex=True, figsize=(7, 6),
    gridspec_kw={"height_ratios": [3, 1]},
    constrained_layout=True,
)
ax.errorbar(inverse_v, inverse_b, yerr=sigma_inverse_b,
            fmt="o", capsize=3, label="Metingen")
ax.plot(x_plot, lineair(x_plot, f_fit), label="Fit")
ax.set_ylabel("1/b (1/m)")
ax.legend()

ax_res.errorbar(inverse_v, residuen, yerr=sigma_inverse_b,
                fmt="o", capsize=3)
ax_res.axhline(0, color="black", linestyle="--")
ax_res.set_xlabel("1/v (1/m)")
ax_res.set_ylabel("Residu (1/m)")
plt.show()
```

Controleer voordat je een conclusie trekt:

1. Loopt de fit door de meetpunten, rekening houdend met de foutbalken? Niet ieder punt hoeft binnen één foutbalk van de lijn te liggen.
2. Liggen de residuen zonder duidelijk patroon rond nul? Kromming of een trend kan wijzen op een ongeschikt model of een systematisch effect.
3. Is de gevonden brandpuntsafstand fysisch aannemelijk? Bespreek ook effecten die niet in `b_err` zitten, zoals een verkeerd gekozen nulpunt.

Wat minimaliseert de fit?

Met $x_i = \frac{1}{v_i}$, $y_i = \frac{1}{b_i}$ en $\sigma_{y,i} = \frac{\sigma_{b,i}}{b_i^2}$ kiest de gewogen kleinste-kwadratenfit de parameter f die de volgende som minimaliseert:

$$\chi^2(f) = \sum_i \left(\frac{y_i - \left(-x_i + \frac{1}{f}\right)}{\sigma_{y,i}} \right)^2 .$$

Zo telt een afwijking relatief aan de onzekerheid mee. Bij niet-lineaire modellen is de onzekerheid uit `pcov` een lokale benadering rond de gevonden oplossing.
Meer over de argumenten en uitvoer: SciPy-documentatie van `curve_fit`.

Appendix: Materialen Lijst Practica

- Onbekend Lens (1x)
- Optische rail
- Dia-houders + dia (letter F)
- Projectiescherm
- Rolmaat
- Ruiters 3 hoog, 1 laag.
- LED bron, 2 kabels en voedingsbron.